
Identification persistante des ports série

Linux (udev) & Windows — Guide synthétique pratique

Document généré le 21/09/2026

1. Introduction

Lorsqu'on branche plusieurs convertisseurs USB-série (ou des équipements USB présentant une interface série), le noyau Linux attribue des noms dynamiques (/dev/ttyUSB0, /dev/ttyACM0...) et Windows attribue des numéros de port COM. Ces noms peuvent changer selon l'ordre de branchement ou après un redémarrage. Pour garantir une association stable à un équipement donné, on s'appuie sur le **numéro de série USB** du périphérique.

2. Sous Linux — Règles udev

2.1 Récupérer les attributs du périphérique

Branchez le périphérique puis exécutez :

```
udevadm info -a -n /dev/ttyUSB0 | grep -E 'idVendor|idProduct|serial'
```

Exemple de sortie utile :

```
ATTRS{idVendor}=="0403"  
ATTRS{idProduct}=="6001"  
ATTRS{serial}=="A6008isP"
```

2.2 Créer une règle udev personnalisée

Créez le fichier /etc/udev/rules.d/99-usb-serial.rules :

```
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001",  
ATTRS{serial}=="A6008isP", SYMLINK+="ttyMonEquipement"
```

Rechargez les règles :

```
sudo udevadm control --reload-rules  
sudo udevadm trigger
```

Le lien symbolique /dev/ttyMonEquipement pointera désormais toujours vers le bon périphérique.

2.3 Liens automatiques fournis par le système

La plupart des distributions créent automatiquement des liens persistants dans :

```
/dev/serial/by-id/  
/dev/serial/by-path/
```

Exemple : /dev/serial/by-id/usb-FTDI_FT232R_USB_UART_A6008isP-if00-port0. Ces liens sont basés sur le numéro de série et peuvent être utilisés directement sans règle personnalisée.

3. Python sous Linux — Retrouver les liens

3.1 Symlink → périphérique réel

```
from pathlib import Path  
  
def symlink_to_device(symlink_path: str) -> str:
```

```

path = Path(symlink_path)
if not path.is_symlink():
    raise ValueError(f"{symlink_path} n'est pas un lien symbolique")
return str(path.resolve())

# Exemple
print(symlink_to_device("/dev/serial/by-id/usb-..."))
# → /dev/ttyACM0

```

3.2 Périphérique → liens symboliques

```

from pathlib import Path

def device_to_symlinks(device: str, search_dirs=None) -> list[str]:
    if search_dirs is None:
        search_dirs = ["/dev/serial/by-id", "/dev/serial/by-path", "/dev"]
    device = Path(device).resolve()
    matches = []
    for directory in search_dirs:
        dir_path = Path(directory)
        if not dir_path.is_dir():
            continue
        for entry in dir_path.iterdir():
            if entry.is_symlink():
                try:
                    if entry.resolve() == device:
                        matches.append(str(entry))
                except (OSError, RuntimeError):
                    continue
    return sorted(matches)

print(device_to_symlinks("/dev/ttyACM0"))

```

4. Sous Windows

Il n'existe pas de liens symboliques équivalents à `/dev/serial/by-id/`. Les ports s'appellent COMx. Windows mémorise généralement le numéro de COM associé à un numéro de série USB donné (mécanisme de « COM Port Retention »).

4.1 Forcer un numéro de COM fixe

1. Ouvrir le **Gestionnaire de périphériques** (devmgmt.msc)
2. Développer **Ports (COM et LPT)**
3. Clic droit sur le port → **Propriétés** → onglet **Paramètres du port** → **Avancé**
4. Choisir le numéro de COM souhaité dans la liste déroulante

Si le périphérique possède un numéro de série unique, Windows conserve généralement cette association même après redémarrage ou changement de port USB.

4.2 Python sous Windows (pyserial)

La bibliothèque **pyserial** fournit toutes les informations nécessaires (numéro de série, VID, PID, emplacement, etc.) :

```

import serial.tools.list_ports

def find_port_by_serial(serial_number: str) -> str | None:
    """Retourne le port COM à partir du numéro de série."""
    for port in serial.tools.list_ports.comports():
        if port.serial_number and port.serial_number.upper() == serial_number.upper():
            return port.device # ex: "COM5"
    return None

def find_serial_by_port(com_port: str) -> str | None:
    """Retourne le numéro de série à partir du port COM."""
    for port in serial.tools.list_ports.comports():

```

```

        if port.device.upper() == com_port.upper():
            return port.serial_number
    return None

```

```

# Exemples
print(find_port_by_serial("A6008isP"))    # → "COM7"
print(find_serial_by_port("COM7"))        # → "A6008isP"

```

Chaque objet `ListPortInfo` expose notamment : `device`, `serial_number`, `vid`, `pid`, `location`, `description`, `hwid`.

5. Tableau comparatif

Aspect	Linux	Windows
Nom du port	/dev/ttyUSBx, /dev/ttyACMx	COMx
Liens persistants	/dev/serial/by-id/ et by-path/	Aucun (le COM est le nom)
Mécanisme principal	Règles udev + numéro de série	COM Port Retention + serial
Forcer un nom fixe	SYMLINK+="mon_nom"	Gestionnaire de périphériques
Outil Python recommandé	pathlib + os	pyserial (list_ports)
Identification fiable	serial + idVendor + idProduct	serial_number + vid + pid

6. Bonnes pratiques

- **Toujours combiner** numéro de série + VID + PID pour une identification robuste.
- Préférer les liens système (/dev/serial/by-id/ ou pyserial) plutôt que de hardcoder des noms volatils.
- Sur les adaptateurs bon marché sans numéro de série (certains CH340), utiliser le chemin physique USB (moins portable).
- Tester après un redémarrage et un changement de port USB pour valider la persistance.
- Documenter le numéro de série de chaque équipement dans votre inventaire.

7. Résumé express

Linux : créez une règle udev basée sur `ATTRS{serial}` ou utilisez directement `/dev/serial/by-id/...`. En Python, `Path.resolve()` et un parcours des répertoires de liens suffisent.

Windows : le numéro de série USB permet à Windows de conserver le même COM. En Python, `serial.tools.list_ports` donne immédiatement le mapping `serial ↔ COMx`.

Ce document reprend les méthodes les plus fiables et les plus utilisées pour associer de façon permanente un port série à un équipement identifié par son numéro de série USB.